مهندسی نرم‌افزار

جلسهٔ ۴ — مدل‌سازی و معماری

ابراهیم فضلی

ترم ۱۴۰۴

مرور جلسهٔ ۳

- فرآیند = نقشهٔ راه تیم
- Waterfall، Scrum Agile
- نقش‌ها، رویدادها، آرتیفکت‌ها
- Standup Daily
- Kanban و XP
- تسک: ۱۰ Backlog + Story User

چرا مدل‌سازی مهم است؟

مدل = ساده‌سازی واقعیت

قبل از کدنویسی، سیستم را طراحی می‌کنیم.

مدل خوب = کد بهتر، تغییرات آسان‌تر.

UML چیست؟

Language Modeling Unified

- زبان استاندارد مدل‌سازی نرم‌افزار
- ۱۴ نوع دیاگرام
- مهم‌ترین‌ها:
  - Diagram Case Use
  - Diagram Class
  - Diagram Sequence
  - Diagram Activity

Diagram Case Use

چه کسی با سیستم چه کاری می‌کند؟

- Actor: کاربر یا سیستم خارجی
- Case: Use یک عملکرد
- Relationship: ارتباط

مثال: مشتری مشاهدهٔ محصولات

Diagram Class

ساختار کلاس‌ها و روابط

- Class: نام، ویژگی‌ها، متدها
- Association: ارتباط ساده
- Inheritance: ارث‌بری
- Composition: ترکیب

مثال: Customer Order، Product،

Diagram Sequence

ترتیب تعاملات در زمان

- Actor: کاربر
- Object: شیء
- Message: پیام
- Lifeline: خط زمان

مثال: مشتری سبد خرید ثبت سفارش

Diagram Activity

جریان کار و تصمیم‌ها

- End: / Start شروع و پایان
- Action: فعالیت
- Decision: تصمیم
- Join: / Fork انشعاب

مثال: فرآیند خرید

نمونه: UML در HomeStore

دیاگرام	کاربرد
Case Use	خرید، مدیریت، انبار
Class	Customer Order، Product،
Sequence	ثبت سفارش
Activity	فرآیند پرداخت

ابزارهای UML

- draw.io — رایگان، آنلاین
- Lucidchart — حرفه‌ای، رایگان محدود
- PlantUML — متنی، برای کدنویسی
- StarUML — حرفه‌ای، دسکتاپ
- Figma — طراحی، نه UML خالص

توصیه: draw.io یا PlantUML

معماری نرم‌افزار

ساختار کلی سیستم

- معماری: تصمیم‌های سطح بالا
- طراحی: جزئیات پیاده‌سازی
- معماری خوب = تغییرات آسان‌تر

معماری = نقشهٔ ساختمان، طراحی = نقشهٔ اتاق‌ها

معماری لایه‌ای

Architecture Layers

لایه	مسئولیت
Presentation	تعامل با کاربر، UI
Business	منطق کسب‌وکار
Data	دسترسی به دیتابیس

مزیت: جدایی مسئولیت‌ها

Controller View Model – MVC

الگوی طراحی UI

- Model: داده و منطق
- View: نمایش
- Controller: واسط بین Model و View

محبوب در فریم‌ورک‌های وب

Microservice vs Monolith

	Monolith	Microservice
ساختار	یک برنامه	چند سرویس
استقرار	یکجا	جداگانه
مقیاس	کلی	تکی
پیچیدگی	کم	زیاد
مناسب	تیم کوچک	تیم بزرگ

API REST

واسط بین Frontend و Backend

Method	کاربرد
GET	خواندن
POST	ایجاد
PUT	به‌روزرسانی
DELETE	حذف

مثال: /api/products

نمونه: معماری HomeStore

لایه	فناوری
Presentation	React
Business	Express + Node.js
Data	PostgreSQL

- SPA Frontend:
- API REST Backend:
- Relational Database:

Trade-offهای معماری

- سادگی vs انعطاف
- سرعت vs مقیاس‌پذیری
- امنیت vs سرعت توسعه
- هزینه vs کیفیت

هیچ معماری‌ای کامل نیست

انتخاب معماری

- تیم کوچک، پروژهٔ ساده: Monolith
- تیم بزرگ، پروژهٔ پیچیده: Microservice
- پروژهٔ محتوایی: Jamstack
- پروژهٔ Real-time: Event-driven

برای HomeStore: Layered + Monolith

مدل داده

ساختار داده‌ها و روابط

- Entity: موجودیت (مثل Product)
- Attribute: ویژگی (مثل price)
- Relationship: رابطه (مثل Product-Order)

Diagram Relationship Entity — ERD

نمودار موجودیت‌ها و روابط

موجودیت	ویژگی‌های کلیدی
Product	stock price، name، id،
Category	name id،
Order	total customer_id، id،
Customer	phone name، id،

نرمال‌سازی

کاهش تکرار داده‌ها

- NF:۱ حذف گروه‌های تکراری
- NF:۲ حذف وابستگی جزئی
- NF:۳ حذف وابستگی انتقالی

معمولاً تا NF۳ کافی است

نمونه: دیتابیس HomeStore

جدول	نقش
products	محصولات
categories	دسته‌بندی‌ها
orders	سفارش‌ها
order_items	اقلام سفارش
customers	مشتریان
users	کاربران (ادمین)

تسک هفتهٔ آینده

تسک شما:

- Diagram Case Use برای پروژه
- Diagram Class اولیه
- انتخاب معماری
- ERD دیتابیس
- مستندسازی تصمیم‌ها

تسک من:

- architecture.md در HomeStore
- database-schema.md
- Diagram Case Use
- Diagram Class
- آماده‌سازی Sprint ۱

سؤالی دارید؟ به امید یک ترم پربار